Durham University

OOP with Classes and Objects

Mladen Ivkovic

COMP2221 Programming Paradigms – Object-oriented Programming

Lecture 1 Recap

When poll is active respond at

PollEv.com /mladenivkovic356

Durham University

First Steps With Java

Hello World

```
// file MyMain.java

class MyMain {

  public static void main(String args[]){
    System.out.println("Hello World!");
  }
}
```

- Java code is stored in a (plain text) file.
- The file must contain a (public) **class**.
- The class must have the same name as the file.
  - (not strictly enforced by all compilers, but good practice.)
- Your "main" class must contain a `main` method.
  - (there are exceptions in modern Java, but let's ignore these for now.)

Hello World – main() method

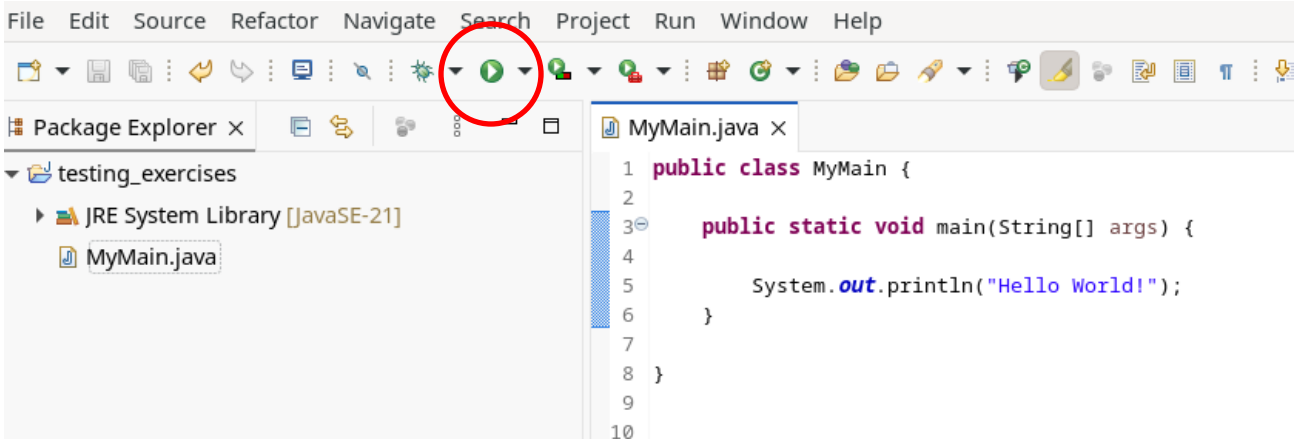
```
// file MyMain.java

class MyMain {

  public static void main(String args[]){
    System.out.println("Hello World!");
  }
}
```

- The *main()* method must:
  - Be *public*
  - Be *static*
  - Have return type *void*
  - Accept an array of *Strings* as the argument

Durham University

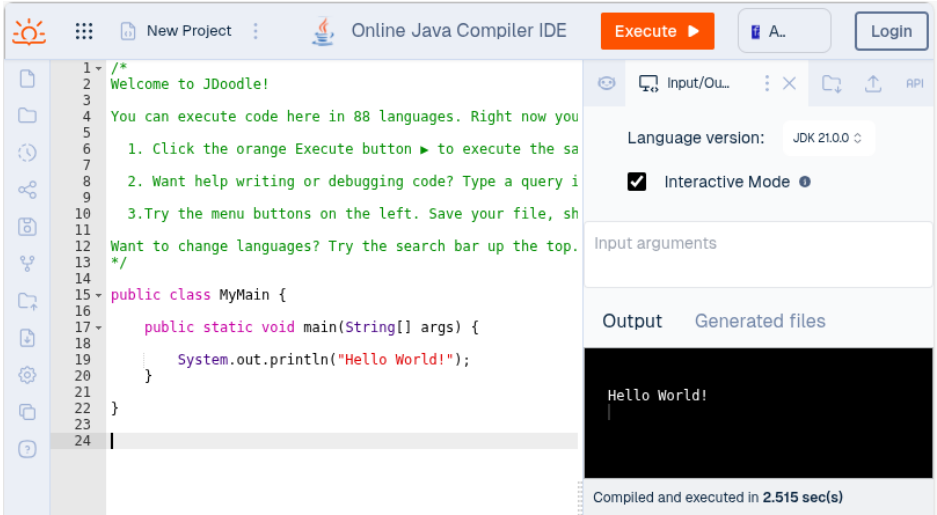
How to run Java - Eclipse

- Eclipse IDE (available on lab Windows PCs through AppsAnywhere)



How to run Java - Online Compiler

- https://www.jdoodle.com/online-java-compiler



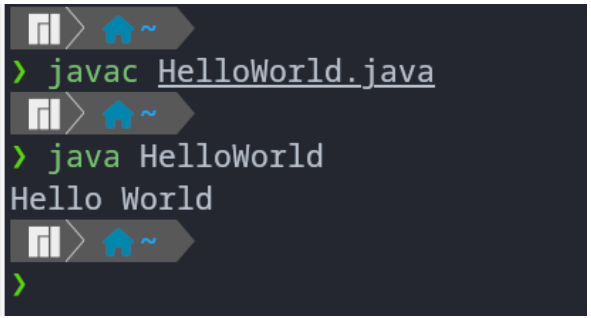
Durham University

How to run Java – Compile Manually

- It can be done, but ***not recommended for this course***
- 1) install Java Development Kit on your machine
- 2) Write your program in a (plain text) file
- 3) Compile the program to bytecode using

  `javac YourFileName.java`
- 4) Run the compiled bytecode using java

  `java YourFileName`



Common Java Data Types

```
int myNum = 5;               // Integer (whole number)
float myFloatNum = 5.99f;    // Floating point nr
char myLetter = 'D';         // Character
boolean myBool = true;       // Boolean
String myText = "Hello";     // String
```

Data types are divided into two groups:

- **Primitive data types**

  Includes **`byte`, `short`, `int`, `long`, `float`, `double`, `boolean`,** and **`char`**

- **Non-primitive data types**

  such as ***`String`***, ***`Arrays`*** and ***`Classes`***

  (you will learn more about these in later slides/lectures)

Branching

```
int a = 10;

if (a < 3) {
    a += 20;
}
```

```
if (a > 3) {
    a -= 300;
}
else if (a == 3) {
    a = 24;
}
else {
    a = 0;
}
```

Durham University

Loops

```
// for loops
for (int i = 0; i < 5; i++) {
  System.out.println("i=" + i);
}

// while loops
int j = 0;
while (j < 10) {
  System.out.println("j=" + j);
  j += 1;
}
```

```
for (int k = 0; k < 10; k++) {
  if (k == 4) {
    // stop iteration and continue
    // with next value of k
    continue;
  }
  if (k == 8) {
    // exit the loop.
    break;
  }
  System.out.println("k=" + k);
}
```

Durham University

Java and OOP – Classes and Objects

What Will You Learn in this Lecture?

- What are the major concepts of object-oriented programming?
  - **Classes**
  - **Objects**
- How to write a class?

Why We Need OOP

- Let's say we are required to build an application: **Animal Farming Game**

Why We Need OOP

- **In the program, we will need to implement the following variables and methods:**

Note: needs to be implemented, this isn't the implementation!

```
• string strCow1Name
• float fCow1Weight
• void DrawCow1()

• string strCow2Name
• float fCow2Weight
• void DrawCow2()

• string strChicken1Name
• float fChicken1Weight
• bool bChicken1LayEgg
• void DrawChicken1()
```

Durham University

There must be a better way

```
• string strCow1Name
• float fCow1Weight
• void DrawCow1()

• string strCow2Name
• float fCow2Weight
• void DrawCow2()

• string strChicken1Name
• float fChicken1Weight
• bool bChicken1LayEgg
• void DrawChicken1()
```

- The program quickly becomes challenging to follow
  - Simple enough with 2 cows and a chicken, but what about four trillion particles?
- We end up copying and pasting code whenever a new animal is needed.
- There must be a better way!
- ***Any ideas?***

There must be a better way

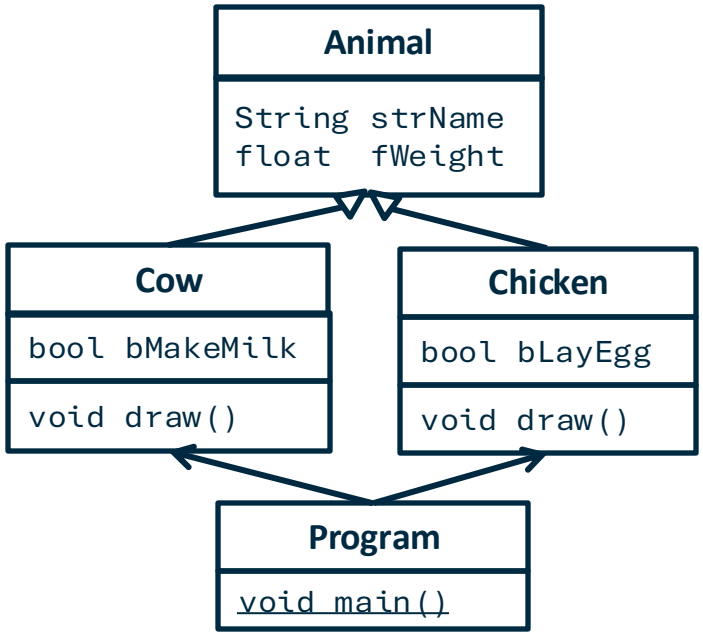
```
• string strCow1Name
• float fCow1Weight
• void DrawCow1()

• string strCow2Name
• float fCow2Weight
• void DrawCow2()

• string strChicken1Name
• float fChicken1Weight
• bool bChicken1LayEgg
• void DrawChicken1()
```

- The program quickly becomes challenging to follow
  - Simple enough with 2 cows and a chicken, but what about four trillion particles?
- We end up copying and pasting code whenever a new animal is needed.
- There must be a better way!
- ***Any ideas?***
- ***Use OOP!***
  - Define blueprints for cows, chickens etc
  - Group data and methods together into blueprint
  - Create an object according to the blueprint when you need it

Durham University

We Can Go Even Further With OOP: Inheritance!

- We can create classes for cows and chickens
- There are things we can re-use (name, weight)
- Re-use a blueprint for other blueprints! Or: Let a class (blueprint) ***inherit*** from other classes (blueprints)
- UML Class Diagram (right): A standard way of visualising classes.
  - We will use it more in future lectures.

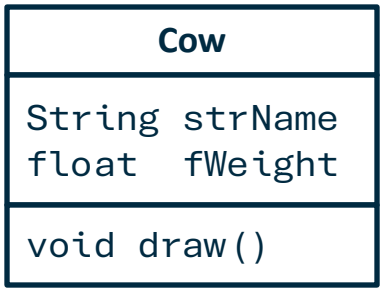


An OOP Framework for the Animal Farm

- **Class** – A blueprint to create something with all relevant variables and methods
  - E.g. A class of "**Cow**"

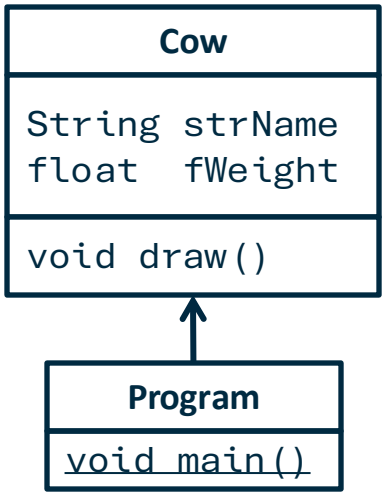
- **Object** – An instance of a class, initialising the computer memory needed to store specific features of that "something"
  - E.g. "**cow1**" and "**cow2**" are objects of the class "Cow"

Class Cow



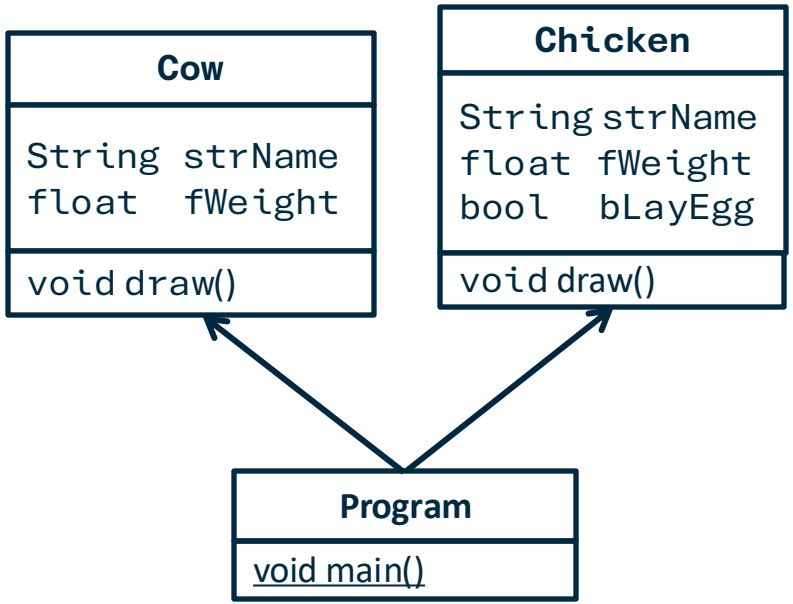
- All cow-related variables and methods in the Cow class

Main Application



- The main application initialises two objects of **Cow**, namely *cow1* and *cow2*
- Using "." to assess the right variable and method
  - *cow1.strName* stores the name of *cow1*
  - *cow2.draw()* call the draw function for *cow2*
- Inside `main()`, we can have
  - *Cow cow1*
  - *Cow cow2*

Visualising the Class Concept



- Similarly, the Chicken class can be implemented as well
- The program should become a lot easier to maintain:
  - ***If we need to add a new feature for all the cows, we only need to change the `Cow` class***
- Inside *`main()`* we can have:
  - *`Cow cow1`*
  - *`Cow cow2`*
  - *`Chicken chicken1`*

Durham University

Programming with Classes and Objects

The Three Parts of A Class

- A class usually contains:
  - **Class variables (i.e. state)**
    - To store the features
    - Also known as “fields”
  - **Class methods (i.e. behaviour)**
    - To perform actions
    - Also known as “functions”
  - **Constructor(s)**
    - To build an object (i.e. an instance) of the class
    - The constructor always has the same name as the class

Example: A Cow Class

```
class Cow {
  // Class variables
  private String name;
  private double weight;

  // Constructor
  public Cow(String name, double weight) {
    this.name = name;
    this.weight = weight;
  }

  // Class methods
  // Getter for name
  public String getName() { return name; }

  // Setter for name
  public void setName(String name) {
    this.name = name;
  }

  // Getter for weight
  public double getWeight() {
    return weight;
  }

  // Setter for weight
  public void setWeight(double weight) {
    this.weight = weight;
  }

  // Method to display cow information
  public void displayInfo() {
    System.out.println("Cow Name: " + name +
                       ", Weight: " + weight + " kg");
  }
} // end of Class Cow
```

Example: A Cow Class

```
class Cow {
  // Class variables
  private String name;
  private double weight;

  // Constructor
  public Cow(String name, double weight) {
    this.name = name;
    this.weight = weight;
  }

  // Class methods
  // Getter for name
  public String getName() { return name; }

  // Setter for name
  public void setName(String name) {
    this.name = name;
  }
```

- class variables and methods starting with keyword "**public**":
  - This means that other classes can access them
  - There are other options, such as "**private**" – we will look into this later

The Constructor

```
class Cow {
  // Class variables
  private String name;
  private double weight;

  // Constructor
  public Cow(String name, double weight) {
    this.name = name;
    this.weight = weight;
  }

  // Class methods
  // Getter for name
  public String getName() { return name; }

  // Setter for name
  public void setName(String name) {
    this.name = name;
  }
```

- The constructor:
  - Defines how to ***instantiate*** (create) and how to ***initialise*** an object of the class
  - Must have ***same name*** as the class
  - Must be `public`
    - i.e. starts with keyword "`public`"
    - Since this method is intended to be called by another class

Creating an Object

- An object is created by calling the class constructor in the following format:

```
// Creating a Cow object
Cow cow1 = new Cow("Bella", 450.5);
```

- Compare:

```
int myInt = 3;
// type name = value;
```

Creating an Object – More than one way!

- Suppose we want to be lazy and to have more convenient ways of instantiating an object.
- Example:
  - We want option to specify many properties.
  - We want option to specify few/no properties and use default values instead.
- How do we do that? ***Any ideas?***

Creating an Object – More than one way!

- Suppose we want to be lazy and to have more convenient ways of instantiating an object.
- Example:
  - We want option to specify many properties.
  - We want option to specify few/no properties and use default values instead.
- How do we do that? ***Any ideas?***
- ***We can write more than one constructor!***

Durham University

Method Overloading

Method Overloading

- Polymorphism to the rescue!
- A method has different declarations with different parameter inputs
- The system picks the correct method to run based on the parameters provided

```
// Constructor 1
public Cow(String name, double weight) {
    this.name = name;
    this.weight = weight;
}

// Constructor 2
public Cow() {
    // we could also leave this constructor empty if we wanted...
   this.name = "No Name";
   this.weight = 0.;
}
```

Two constructors:

- the **same method name**
- but different signatures (i.e. different parameters)

Durham University

Calling An Overloaded Method

```
// Creating a Cow object
Cow cow1 = new Cow("Bella", 450.5);

// Creating another Cow object with a different constructor
Cow cow2 = new Cow();
```

Method Overloading: Example 2

```
class Calculator {

  // Method to add two integers
  public int add(int a, int b) {
    return a + b;
  }

  // Overloaded method to add three integers
   public int add(int a, int b, int c) {
    return a + b + c;
  }

  // Overloaded method to add two doubles
  public double add(double a, double b) {
    return a + b;
  }
```

```
  public static void main(String[] args) {

    // get a calculator object
    Calculator calc = new Calculator();

    // Calls add(int, int)
    System.out.println(calc.add(2, 3));

    // Calls add(int, int, int)
    System.out.println(calc.add(2, 3, 4));

    // Calls add(double, double)
    System.out.println(calc.add(2.5, 3.5));
  }

} // end of class Calculator
```

Durham University

Object and Reference

Example: Creating Cow Objects

```
public class MyMain {

  public static void main(String[] args) {
    // Creating a Cow object
    Cow cow1 = new Cow("Bella", 450.5);
    Cow cow2 = new Cow();

    // Displaying cow information
    cow1.displayInfo();

    // Modifying cow properties
    cow1.setName("Daisy");
    cow1.setWeight(500.0);

    // Displaying updated cow information
    cow1.displayInfo();
  }
}
```

Durham University

Example: Creating Cow Objects

```
public class MyMain {

  public static void main(String[] args) {
    // Creating a Cow object
    Cow cow1 = new Cow("Bella", 450.5);
    Cow cow2 = new Cow();

    // Displaying cow information
    cow1.displayInfo();

    // Modifying cow properties
    cow1.setName("Daisy");
    cow1.setWeight(500.0);

    // Displaying updated cow information
    cow1.displayInfo();
  }
}
```

- The object cow1 can be renamed !
- Remember: the Cow class can have different constructors:

```
Cow cow1 = new Cow("Bella", 450.5);
Cow cow2 = new Cow();
```

- We have 2 instances of Cow here
- But we could ***also*** have 2 variables referencing the same object!
  - (this is something we can ***also*** do, this does not replace 2 different constructors)

Durham University

Object and Reference

```
Cow cow1 = new Cow();
```

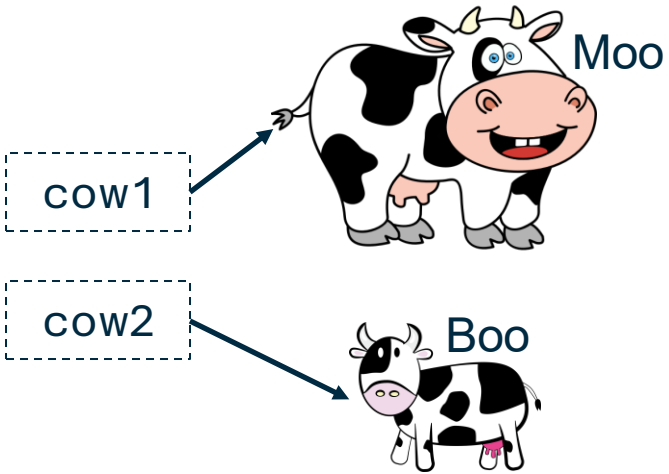
Creates a variable that references to a Cow object

Creates the actual object using the class constructor (e.g. using computer memory to create class variables)

Together, this line of code creates the object and links it to the reference variable

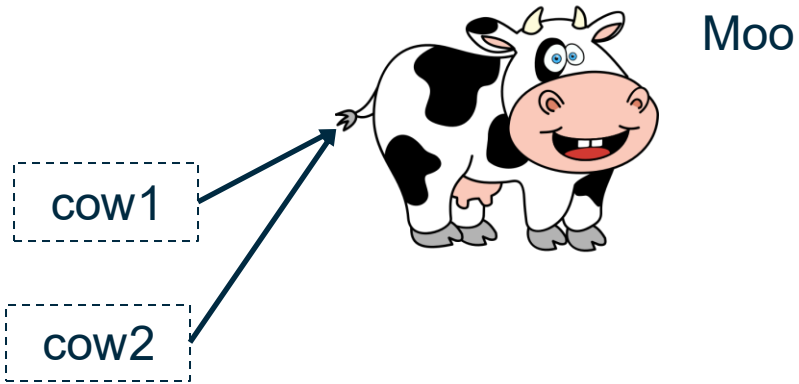
Visualising the Code – Object Creation

```
Cow cow1 = new Cow("Moo", 40.0f);
Cow cow2 = new Cow("Boo", 20.0f);
```



Visualising the Code – Reference Copying

```
Cow cow1 = new Cow("Moo", 40.0f);
Cow cow2 = cow1;
```



Quick Question

- What is the output of the following code?

```
Cow cow1 = new Cow("Moo", 40.0f);
Cow cow2 = cow1;
cow2.setName("Boo");

// Displaying updated cow information
cow1.displayInfo();
cow2.displayInfo();
```

```
// Reminder (this is truncated code from
before):


public Cow(String name, double weight) {
  this.name = name;
  this.weight = weight;
}

public void setName(String name) {
  this.name = name;
}

public void displayInfo() {
  System.out.println("Cow Name: " + name +
            ", Weight: " + weight + " kg");
}
```

Durham University

Quick Question

- What is the output of the following code?

```
Cow cow1 = new Cow("Moo", 40.0f);
Cow cow2 = cow1;
cow2.setName("Boo");

// Displaying updated cow information
cow1.displayInfo();
cow2.displayInfo();
```

Answer:
Cow Name: Boo, Weight: 40.0 kg
Cow Name: Boo, Weight: 40.0 kg

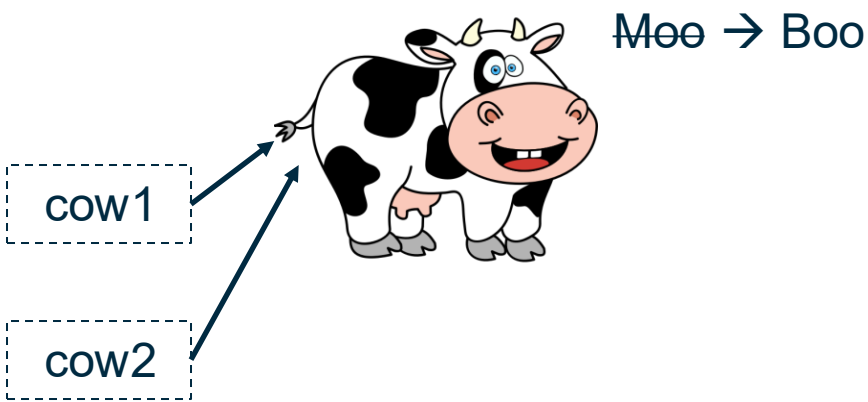
```
// Reminder (this is truncated code from before):


public Cow(String name, double weight) {
  this.name = name;
  this.weight = weight;
}

public void setName(String name) {
  this.name = name;
}

public void displayInfo() {
  System.out.println("Cow Name: " + name +
            ", Weight: " + weight + " kg");
}
```

Durham University

Quick Question

- What is the output of the following code?

```
Cow cow1 = new Cow("Moo", 40.0f);
Cow cow2 = cow1;
cow2.setName("Boo");

// Displaying updated cow information
cow1.displayInfo();
cow2.displayInfo();
```



Answer:
Cow Name: Boo, Weight: 40.0 kg
Cow Name: Boo, Weight: 40.0 kg

Durham University

Summary

- Object-oriented programming
  - Class
  - Object
- Writing a class
  - Class variables
  - Class methods
  - Constructor
- Method overloading
- Object vs. reference

Durham University

Durham University

The End

Any Questions?